CS184/284A Fall 2026 Homework 2: MeshEdit

Team 19

Webpage: https://cal-cs184-student.github.io/hw2-19-writeup/

GitHub repository: https://github.com/cal-cs184-student/hw2-19

PDF version

Overview

This homework goes from curves to meshes. Section I evaluates Bezier curves and surfaces with de Casteljau's algorithm, which only ever blends two points at a time. Section II works on triangle meshes stored as a half-edge data structure: area-weighted vertex normals, edge flips, edge splits, and finally Loop subdivision, which is built out of the flips and splits. All of the code is in `student_code.cpp`.

The biggest lesson was how much care the half-edge pointers need. A flip or split only touches a handful of elements, but if one pointer is wrong the mesh breaks somewhere else later. Drawing every halfedge with a name before writing code, and setting every pointer even when it does not change, made this manageable. I also wrote a separate checker program (not part of the submission) that walks the whole mesh after each operation and confirms every pointer is consistent. It also runs random flips and splits and compares my Loop subdivision against a second, independent implementation. The checker found one real bug in my flip, described in Part 4.

Extra credit done: **boundary edge splits** (Part 5) and **Loop subdivision on meshes with boundary** (Part 6).

All screenshots come from the `meshedit` window. To make them repeatable, I sent the program its normal key and mouse events from a small script and read the window's frame buffer, so the framerate counter is not in the pictures. Edges were picked by hovering and clicking them, then pressing F, S or L, as in the GUI.

Section I: Bezier Curves and Surfaces

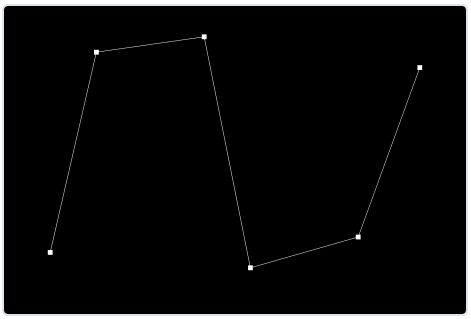
Part 1: Bezier curves with 1D de Casteljau subdivision

de Casteljau's algorithm finds the point at parameter t on a Bezier curve using only linear interpolation. Take each pair of neighboring control points and replace it with the point a fraction t of the way between them:

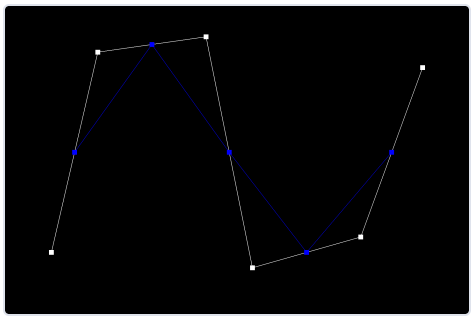
> $$p_i' = \mathrm{lerp}(p_i, p_{i+1}, t) = (1 - t)\, p_i + t\, p_{i+1}$$

n points give $n - 1$ points on the next level. Doing it again gives $n - 2$, and so on, until one point is left. That last point is on the curve. `BezierCurve::evaluateStep` does exactly one level: it loops over $i = 0 \ldots n - 2$ and pushes the blended point into a new vector, using the class member t. Pressing E calls it one more time, and drawing the full curve calls it $n - 1$ times for each value of t.

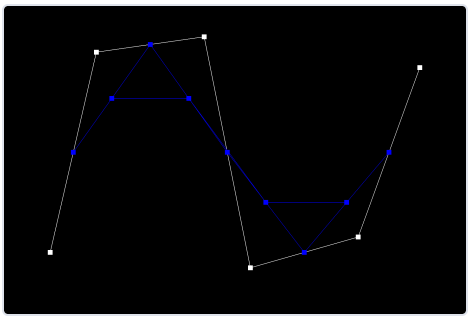
My curve is `bzc/my_curve.bzc` with 6 control points: (0.15, 0.20), (0.30, 0.85), (0.65, 0.90), (0.80, 0.15), (1.15, 0.25), (1.35, 0.80). White squares are the control points, blue are the in-between points, and red is the final point, all at $t = 0.5$.



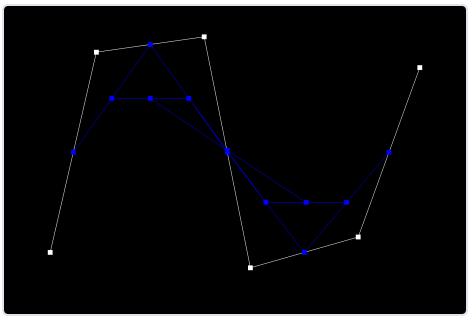
Level 0: the 6 control points



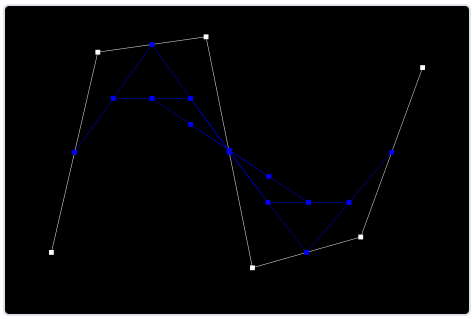
Level 1: 5 points



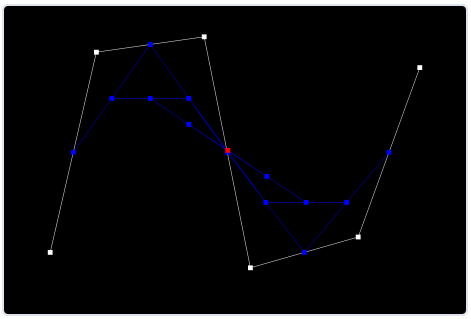
Level 2: 4 points



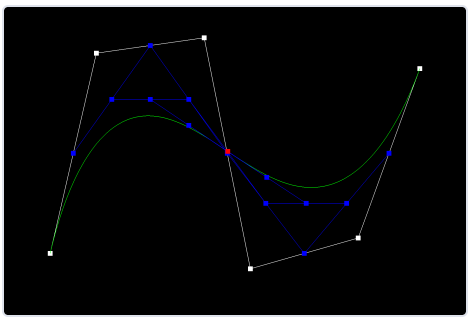
Level 3: 3 points



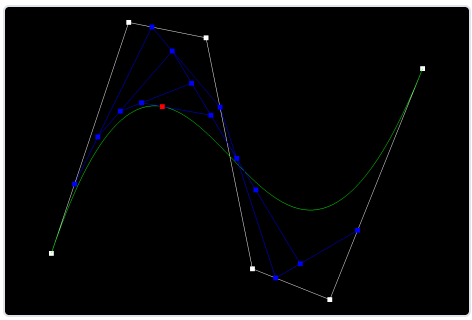
Level 4: 2 points



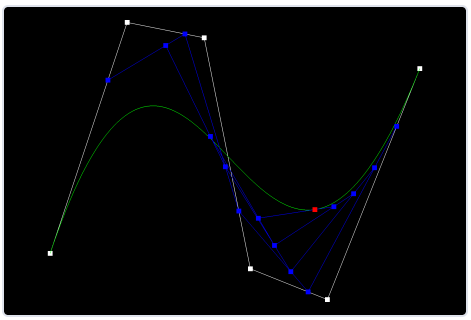
Level 5: 1 point (red), on the curve



All levels with the curve shown (C), $t = 0.5$



Control points 2 and 5 dragged, then scrolled to $t = 0.30$



Same curve, scrolled to $t = 0.75$

The red point always lands on the green curve, and the last blue segment just touches the curve there, because it is the curve's tangent at that point. Moving a control point bends the curve toward it but the curve still starts and ends at the first and last control points.

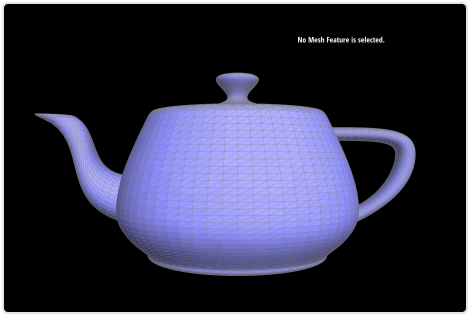
Part 2: Bezier surfaces with separable 1D de Casteljau

A bicubic Bezier patch has a 4×4 grid of control points P_{ij}, and the surface is

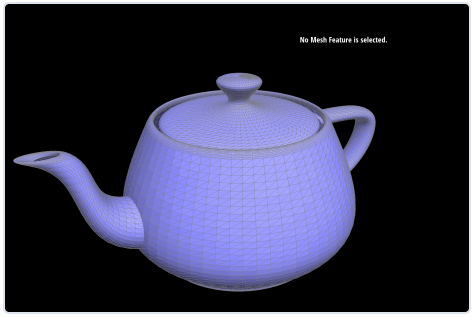
> $$S(u, v) = \Sigma_i \Sigma_j B_i(v) B_j(u) P_{ij}$$

This is separable. If *v* is held fixed, each row of 4 control points is just a Bezier curve in *u*. So the 1D algorithm works twice: first run 1D de Casteljau on each row with *u*, which gives 4 points (one per row). Those 4 points make a curve in *v*, and running 1D de Casteljau on them with *v* gives the point on the surface.

`BezierPatch::evaluateStep` is the same step as Part 1 but for 3D points, with *t* passed in. `evaluate1D` keeps calling it until one point is left. `evaluate(u, v)` calls `evaluate1D` on each row with *u*, then on the 4 results with *v*. The viewer samples each patch on a 16×16 grid and makes two triangles per cell. For `bez/teapot.bez` that comes to 8,258 vertices and 16,256 triangles after duplicate vertices are merged.



`bez/teapot.bez` from my implementation



Rotated to show the lid and spout opening



Same view in shader mode (W) with smooth normals (Q)

Section II: Triangle Meshes and Half-Edge Data Structure

Part 3: Area-weighted vertex normals

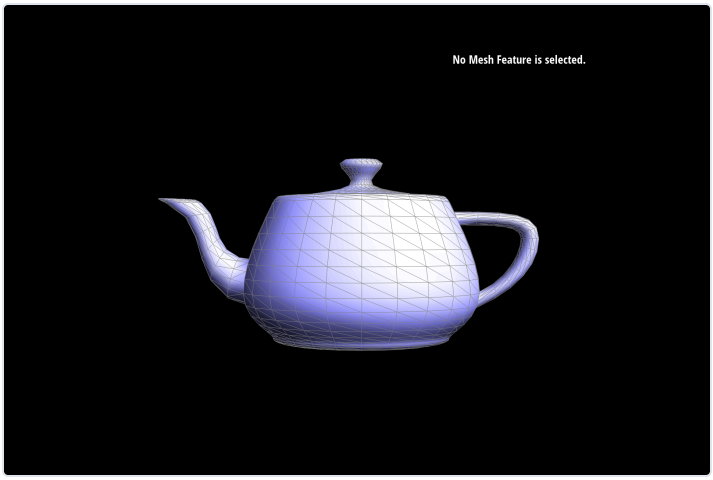
To visit every triangle around a vertex, I start at its halfedge and step with `h = h->twin()->next()` until I am back at the start. Each step gives the next halfedge leaving the vertex. I skip the virtual boundary face. For each real triangle the corners are p_0 (the vertex), p_1 (start of `h->next()`) and p_2 (start of `h->next()->next()`).

> $$n = \text{normalize}\left(\Sigma_{\text{faces}} (p_1 - p_0) \times (p_2 - p_0) \right)$$

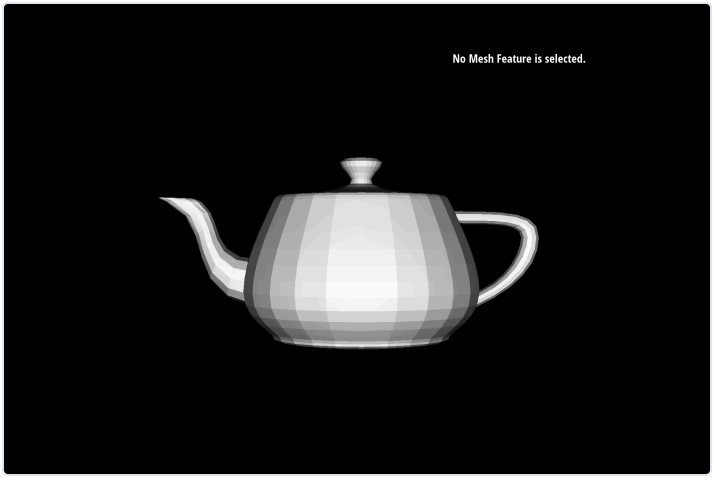
The cross product points along the face normal, and since the corners go counter-clockwise it points out of the surface. Its length is twice the triangle's area. Adding these vectors directly therefore weights each face normal by its area. Normalizing the sum at the end gives the vertex normal. The function is `const`, so it uses `HalfedgeCIter`. My checker compared these normals with a second computation (loop over faces and add each face's area vector to its three corners) on every vertex of all 13 test meshes, and they matched.



`dae/teapot.dae`, default flat shading



After Q: shading uses my vertex normals



Shader mode (W), flat normals: each triangle has one shade



Shader mode (W) + Q: smooth Phong shading

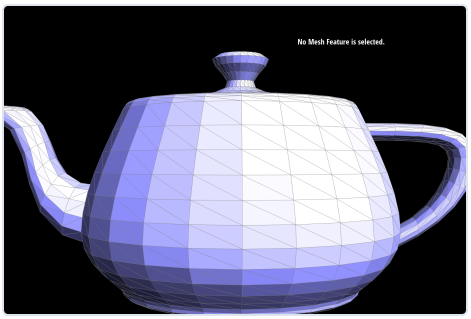
With vertex normals the shading changes smoothly across triangles and the faceted bands go away. The teapot stays bright, which means the normals point outward.

Part 4: Edge flip

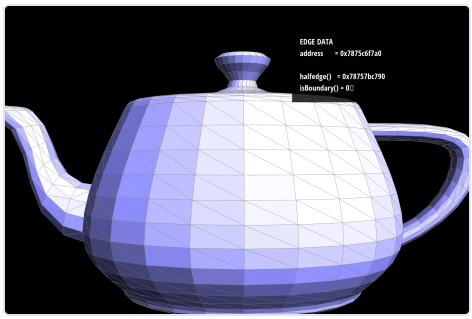
I name everything around the edge first. The edge's halfedge h0 goes v0→v1 inside triangle f0 = (v0, v1, v2), with h1 and h2 after it. Its twin h3 goes v1→v0 inside f1 = (v1, v0, v3), with h4 and h5 after it. The flip turns the shared edge into v2–v3. The new triangles are f0 = (v3, v2, v0) made of h0, h2, h4 and f1 = (v2, v3, v1) made of h3, h5, h1.

- h0 now starts at v3 and h3 at v2. They are still twins and still belong to the same edge.
- The four outer halfedges h1, h2, h4, h5 keep their vertex, edge and twin. Only their `next` and `face` change. h1 and h4 switch faces.
- I reset the halfedge pointer of v0 and v1, since h0 or h3 might have been the one that started at them. I also set v2, v3, both faces and the edge to safe choices.

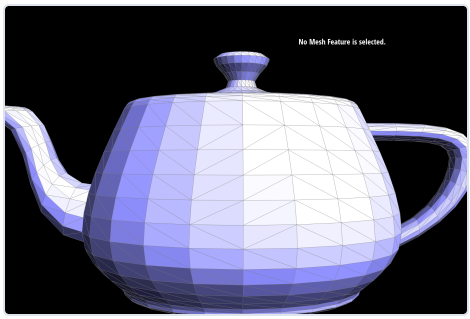
I set every pointer with `setNeighbors`, even the ones that do not change. Boundary edges are returned unchanged. Nothing is created or deleted, and the work is the same for any mesh size.



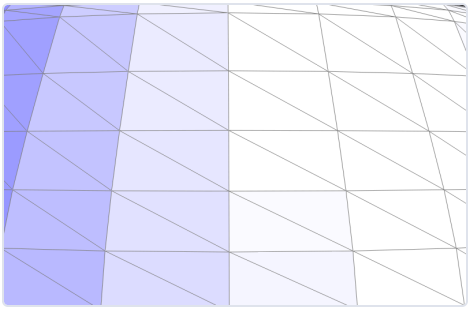
Teapot before flips

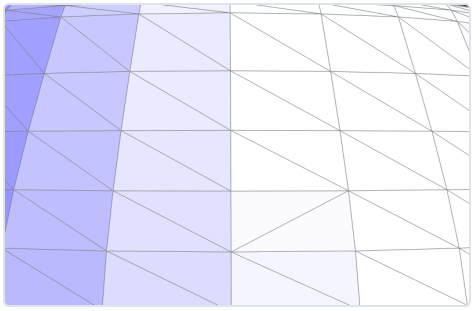


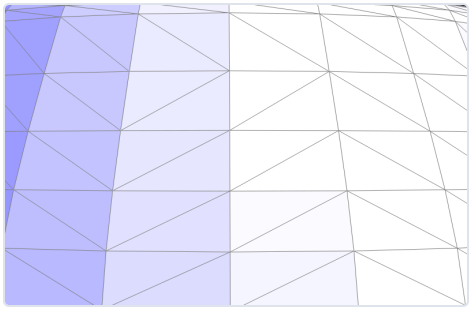
First diagonal selected (HUD shows EDGE DATA)



After flipping 9 diagonals


Close-up: before


Close-up: after 1 flip


Close-up: after 9 flips, a block of diagonals now runs the other way

The vertex, edge and face counts stay at 1,232 / 3,696 / 2,464, and there are no holes. Flipping the same edge twice gives back the original triangles. The checker tested that on hundreds of edges per mesh.

Debugging journey

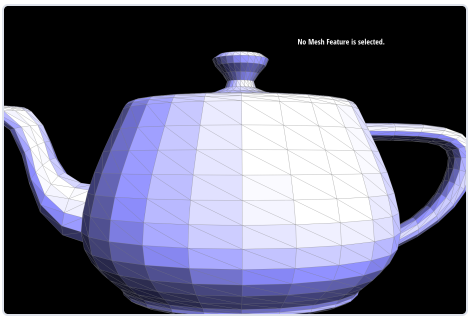
The checker's random test does hundreds of random flips and splits on small meshes and checks every pointer every few steps. It failed after about 250 random operations on the icosahedron and on a flat grid, and after about 410 on the torus. Each time, the failing flip was on an edge whose two sides were the *same* triangle, so v2 and v3 were the same vertex. This only shows up after earlier random flips have already pulled a vertex down to degree 2: then the two triangles beside an edge share their third vertex, and flipping makes an edge from v2 back to itself. One more flip after that broke the `next` pointers. I added a check that leaves an edge alone when v2 == v3. After that, every random run on every mesh stayed valid. Normal meshes never hit this case, but the GUI could, after enough clicking.

Part 5: Edge split

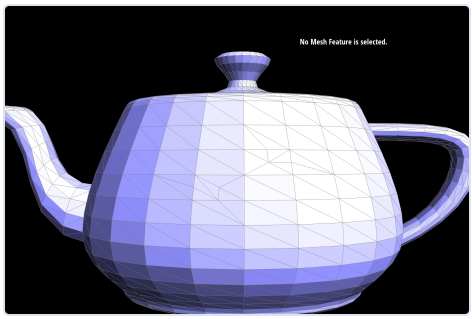
Using the same names as in Part 4, the split adds a vertex m at the midpoint of v0–v1 and connects it to v2 and v3. That needs 1 new vertex, 3 new edges, 2 new faces and 6 new halfedges:

- The old edge keeps the m–v1 half (h0 and h3), and a new edge holds the v0–m half.
- Two new edges cross the old triangles: m–v2 and m–v3.
- f0 becomes (m, v1, v2), a new face is (m, v2, v0), f1 becomes (v1, m, v3), and another new face is (m, v0, v3).
- m's halfedge is h0, which runs along the original edge, as the starter comment asks. v0's halfedge moves to the new v0→m halfedge, since h0 no longer starts at v0.
- For Part 6, m gets `isNew = true`. The two halves of the old edge get `isNew = false`, and the two cross edges get `isNew = true`.

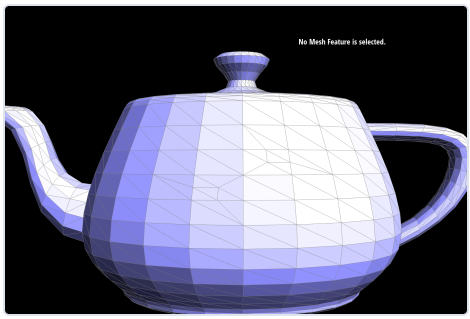
Like the flip, it writes every pointer of every element it touches and does a fixed amount of work. The checker confirmed the counts (+1 vertex, +3 edges, +2 faces) and the midpoint position after every split.



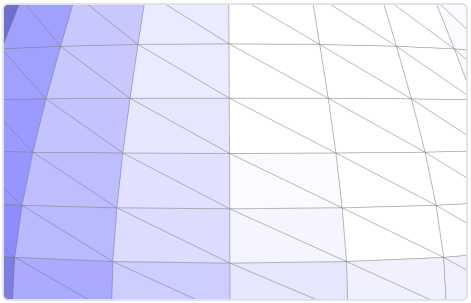
Before

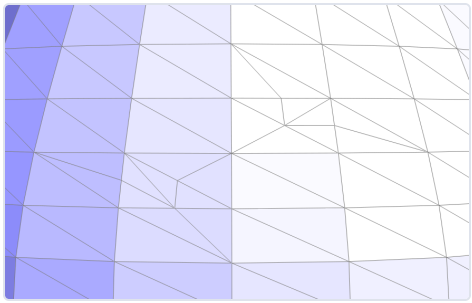


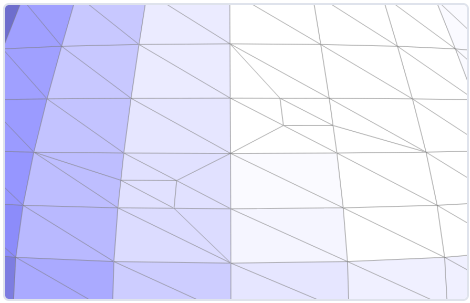
After 6 splits: all three edges of two triangles



After the 6 splits plus 2 flips


Close-up: before

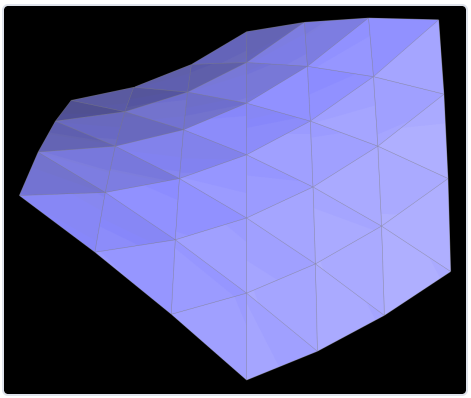

Close-up: each picked triangle has a midpoint on each side


Close-up: one flip each turns them into 4 triangles

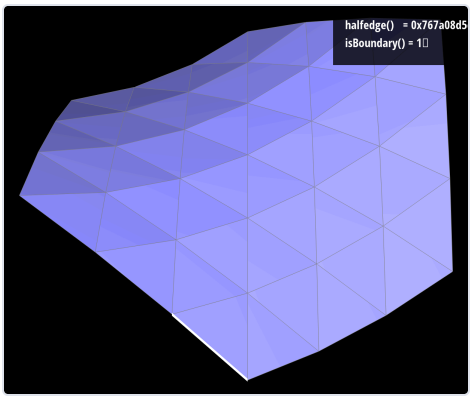
For the splits-plus-flips example, I split all three edges of a triangle (a, b, c). That still leaves an edge from the midpoint of ab to the old corner c. Flipping it joins the other two midpoints instead, so the triangle becomes four smaller ones. That is the same 4-to-1 step Loop subdivision does everywhere in Part 6, done here by hand on two triangles. The counts go from 1,232 / 3,696 / 2,464 to 1,238 / 3,714 / 2,476.

Boundary edges Extra credit

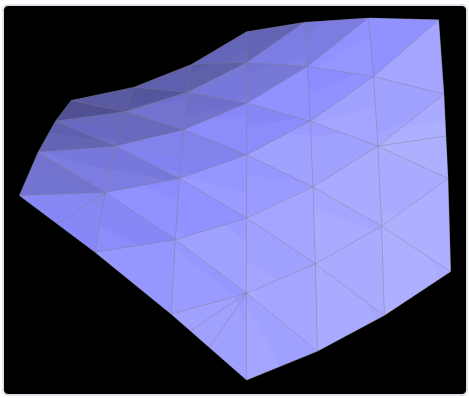
If one side of the edge is the virtual boundary face, I take the halfedge on the real face as h0. Only that triangle gets split: 1 new vertex, 2 new edges, 1 new face, and 4 new halfedges. On the boundary loop, the old boundary halfedge h3 now goes v1→m, and a new boundary halfedge m→v0 is inserted right after it, so the boundary loop gets one edge longer. I tested this on `dae/saddle.dae`, an open hexagonal patch bent into a saddle (37 vertices, 54 triangles, one boundary loop). I made this mesh for these tests.



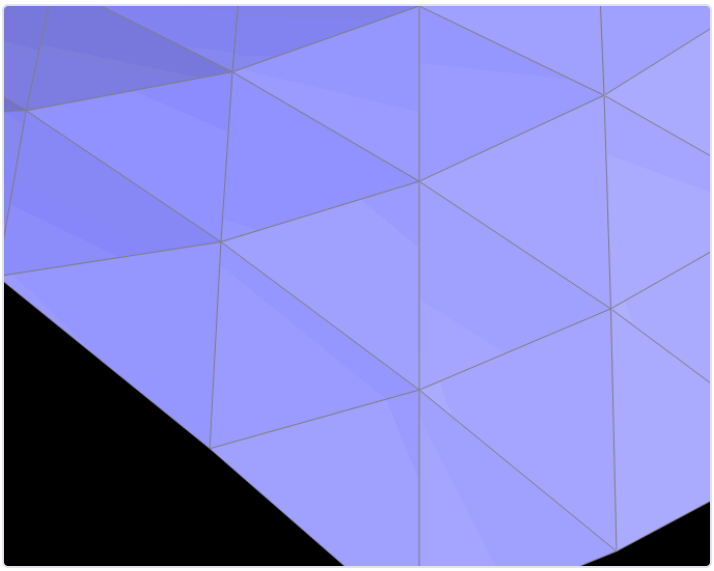
Open saddle mesh before



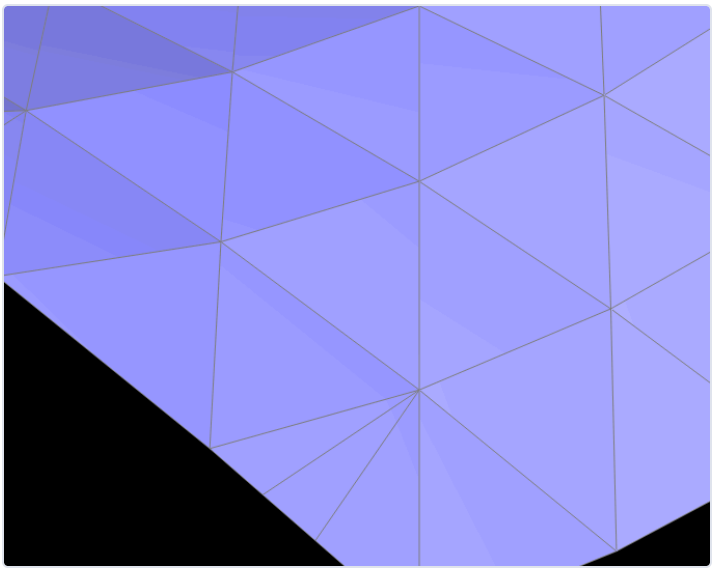
Boundary edge selected (isBoundary() = 1)



After 4 boundary splits: 41 vertices, 58 faces



Close-up before



Close-up after: new boundary vertices connect only into the mesh

Debugging journey

The first time the checker ran on a mesh with a boundary, it crashed. It was following `halfedge()` on the virtual boundary faces, but the starter's `build()` never sets that pointer for boundary faces. My split does not use it either, so the fix was in the checker: it now reaches boundary loops through their halfedges. Separately, one early test sequence flipped the edge from a split midpoint to a quad corner. The new edge then ran straight through the midpoint and left a zero-area triangle, which shows up dark. The pointers were still valid; this is the degenerate case the spec mentions. I replaced that sequence with the 4-to-1 example above.

Part 6: Loop subdivision for mesh upsampling

I compute all new positions on the coarse mesh first, then change the connectivity, then move the vertices:

1. **Old vertices.** Each one gets `isNew = false` and a `newPosition` from its original neighbors. Here n is the number of edges at the vertex, and u = 3/16 when n = 3, or 3/(8n) otherwise:

   > $(1 - n\,u) \cdot \text{original_position} + u \cdot \text{original_neighbor_position_sum}$

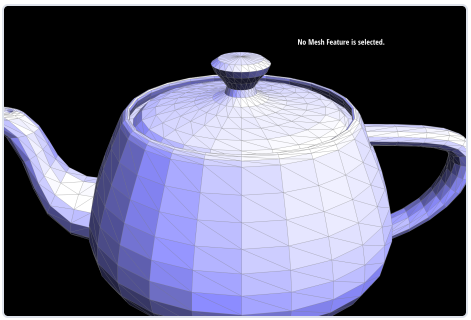
2. **Edges.** Each edge (A, B) between triangles whose third corners are C and D stores the position its new midpoint vertex will get:

   > $3/8\ (A + B) + 1/8\ (C + D)$

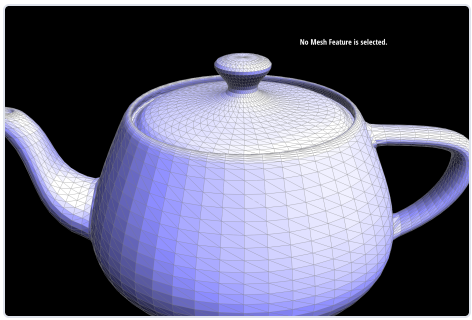
3. **Split.** I first copy every original edge into a list, then split each one. Splits add edges to the end of the mesh's edge list, so looping over the live list would keep finding new edges. Each new vertex copies the stored edge position into its `newPosition`.
4. **Flip.** I flip every edge with `isNew = true` that joins an old vertex to a new one. The edges that are halves of old edges are marked `isNew = false` by the split and are never flipped. New edges between two new vertices are already in the right place.
5. **Move.** Copy `newPosition` into `position` for every vertex.

Computing positions first means the formulas only need to walk the small original mesh, and no position is changed while another vertex still needs the old value. To test it, the checker builds the same subdivision a second way: as a plain list of triangles with a map from each edge to its midpoint index. It then checks that both versions give exactly the same triangles at the same positions, for 1 to 3 rounds on all 13 meshes. Each round also has to give V + E vertices and 4F faces.

How meshes behave



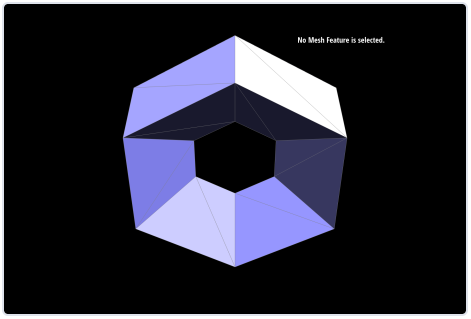
Teapot: 2,464 faces



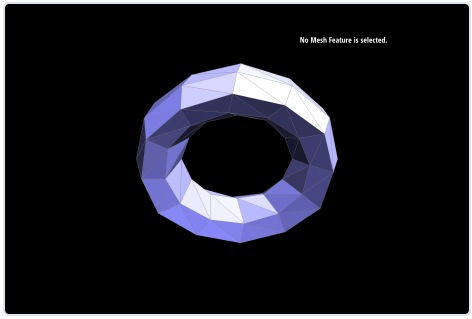
After 1 round: 9,856 faces



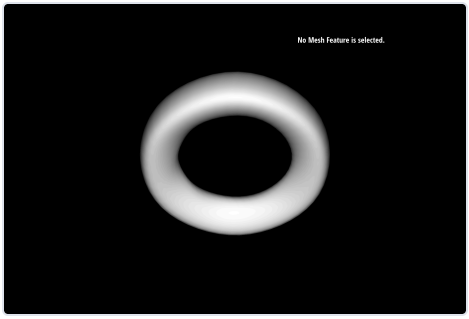
After 2 rounds (39,424 faces), shaded



Coarse torus: 36 faces

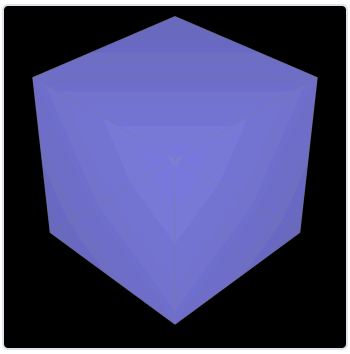


After 1 round

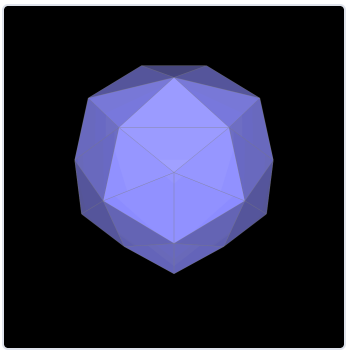


After 3 rounds, shaded

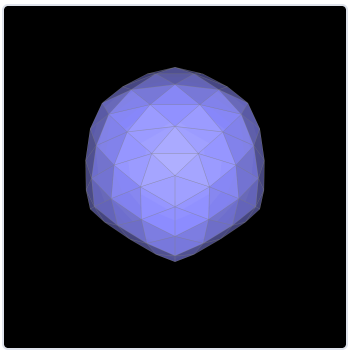
Each round makes the mesh smoother, and big changes happen in the first round or two. Because every vertex moves toward an average of its neighbors, the surface also **shrinks**, most of all at sharp corners and edges. The coarse torus turns into a round ring that is smaller than its first outline. The teapot's lid rim and the edge of its base get softer. A cube shows this the most:



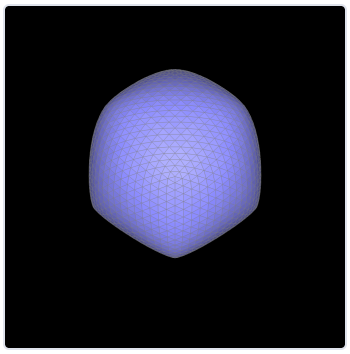
`cube.dae` : 12 faces



1 round: 48 faces



2 rounds: 192 faces

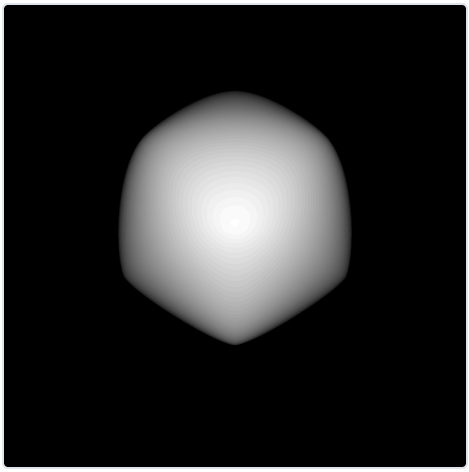


4 rounds: 3,072 faces

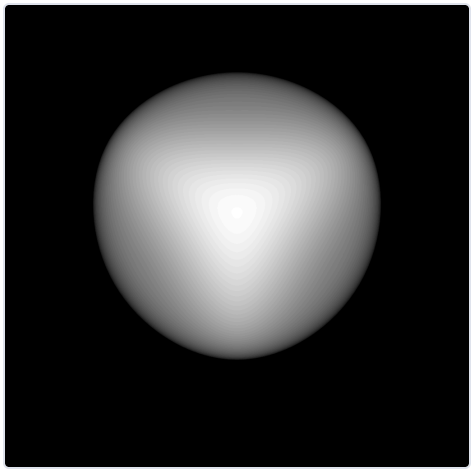
The corners get cut off right away and the cube turns into a lumpy ball. To measure this, I looked at how far the surface reaches from the center toward each of the 8 corners and the 6 face centers. The cube's corners are at (±1, ±1, ±1), so a corner is 1.732 from the center and each face is 1 away. After 4 rounds the faces are only about 0.87 away. The corners are between 0.87 and 1.04 away, depending on which corner.

Pre-splitting edges reduces the effect. Loop subdivision is local: each new position only depends on nearby vertices. Splitting the edges near a sharp feature adds vertices that sit right on it, so after subdivision the surface stays closer to the original shape there. The table uses the same measurements after 4 rounds:

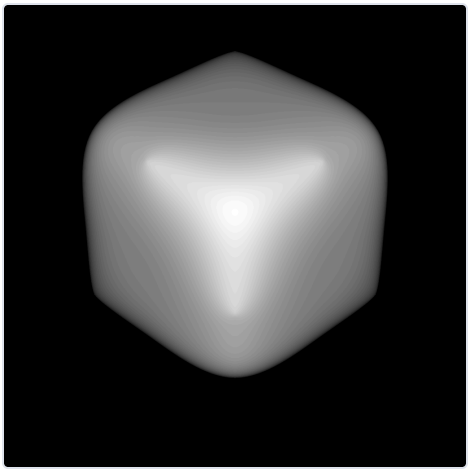
Pre-processing before 4 rounds	Vertices after	Reach toward the 8 corners (cube: 1.732)	Reach toward the 6 faces (cube: 1.0)
none	1,538	0.869 to 1.039 (uneven)	0.863 to 0.879
split the 6 face diagonals	3,074	1.157 at every corner	1.000
split the 6 diagonals, then the 12 cube edges	6,146	1.301 at every corner	1.000
also split each half of the 12 cube edges again	12,290	1.373 at every corner	1.000



No pre-processing, 4 rounds



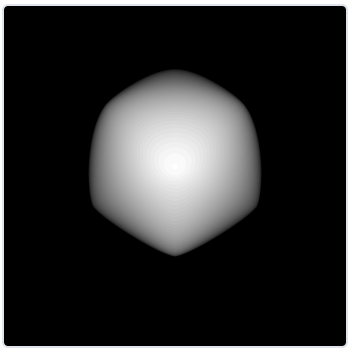
6 diagonals split first, 4 rounds



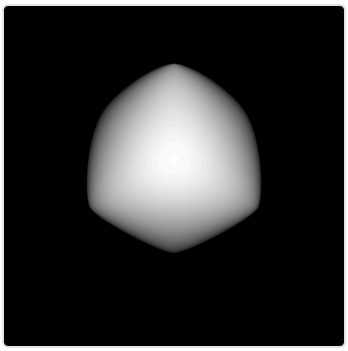
6 diagonals + 12 cube edges split first, 4 rounds: much sharper

The last three shots and the first three table rows were made in the GUI, by selecting and splitting the edges with S before pressing L. The numbers come from the same steps in my checker program, which can read every vertex position. The last table row was done only in the checker.

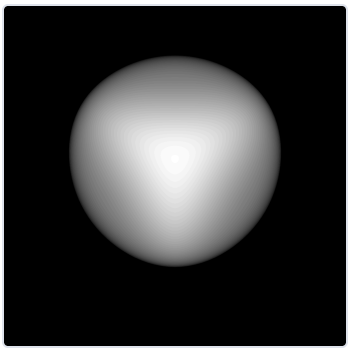
Why the cube comes out lopsided, and the fix



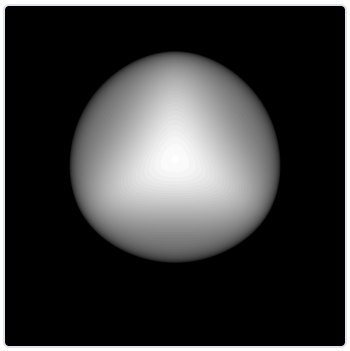
No pre-processing, 4 rounds, viewed from the (+, +, +) corner



Same mesh from the opposite (−, −, −) corner: it looks different



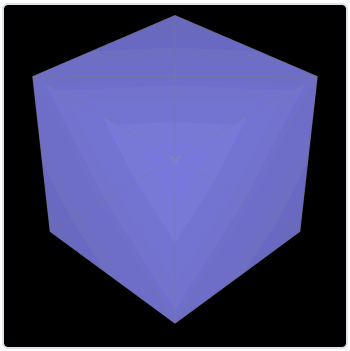
Diagonals split first, 4 rounds, (+, +, +) side



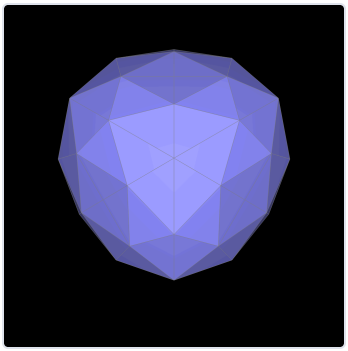
Diagonals split first, (−, −, −) side: now the same

Each square face of `cube.dae` is cut into two triangles by one diagonal, and the diagonals do not follow the cube's symmetry. Three diagonals meet at the corner (−1, −1, −1), so it has degree 6, while (−1, −1, +1) touches no diagonal and has degree 3. The other corners have degree 4 or 5. Loop's weights depend on the degree, and each new point averages the neighbors that happen to be connected. So corners move by different amounts and the faces bend unevenly. The mesh connectivity is not symmetric, so the result is not symmetric either. That is the 0.87 to 1.04 spread in the table.

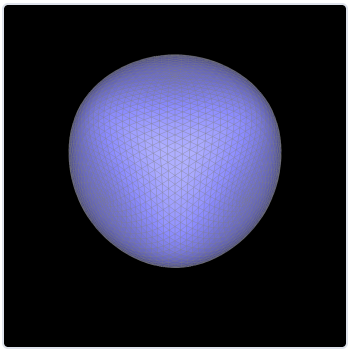
The fix is to **split the 6 face diagonals** before subdividing. Each face then gets a center vertex connected to its 4 corners, so every face is split the same way into 4 triangles. Every corner then has degree 6 and every face center has degree 4. The mesh now has the same symmetry as a cube, so Loop subdivision treats all 8 corners and all 6 faces the same way. All 8 corners end up at exactly 1.157. Splitting the 12 cube edges as well keeps the symmetry and makes the edges sharper.



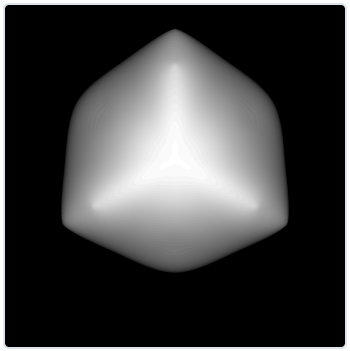
After splitting the 6 diagonals in the GUI: 14 vertices, 24 faces



1 round: every face subdivides the same way



4 rounds: 6,144 faces



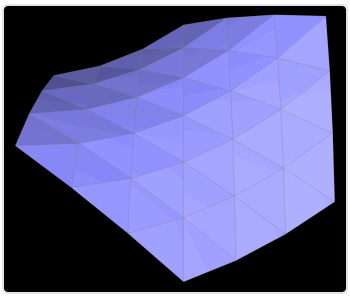
Diagonals + cube edges split, 4 rounds, (−, −, −) side

Meshes with boundary Extra credit

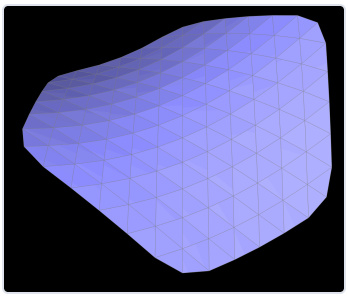
With the boundary split from Part 5 working, the only other change is in the weights. A boundary vertex uses only its two neighbors along the boundary, and a boundary edge's new vertex is just its midpoint:

> boundary vertex: $3/4\ v + 1/8\ (a + b)$ boundary edge: $1/2\ (A + B)$

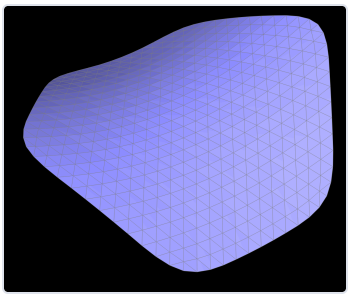
These are the usual Loop boundary rules (the cubic B-spline rules for curves) from the subdivision survey linked in the spec. The boundary turns into a smooth curve that depends only on the boundary vertices, so interior vertices do not pull it around. Its corners still get rounded, as they would on a B-spline curve. Interior vertices next to the boundary use the normal rules. The flip step needs no change, because the new edges are never on the boundary. The checker compared this against the index-based version on the saddle, a flat grid, a bumpy grid, and the starter meshes that have holes (`beetle.dae` has 11 boundary loops and `maxplanck.dae` has 1). A flat square grid stays flat and inside its square.



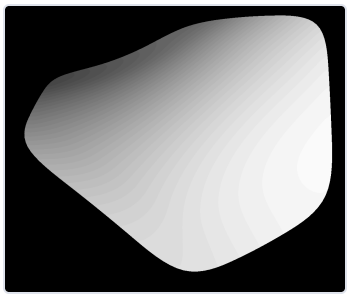
Open saddle: 54 faces



1 round: 216 faces



2 rounds: 864 faces



3 rounds, shaded: smooth, with a clean boundary curve

Section III: Art Competition

I did not enter the optional art competition for this submission.